ТЕХНИЧЕСКИ УНИВЕРСИТЕТ–СОФИЯ

ФАКУЛТЕТ КОМПЮТЪРНИ СИСТЕМИ И ТЕХНОЛОГИИ

КУРСОВА РАБОТА

ДИСЦИПЛИНА: БАЗИ ДАННИ

ТЕМА: УПРАВЛЕНИЕ НА РЕСТОРАНТ

ИЗГОТВИЛ: *Петя Миткова Янева*

ФАКУЛТЕТЕН НОМЕР: *121223006*

СПЕЦИАЛНОСТ: *КОМПЮТЪРНО И СОФТУЕРНО ИНЖЕНЕРСТВО*

КУРС: *II курс*

ГРУПА: *39*

EMAIL: peyaneva@tu-sofia.bg

ТУ - СОФИЯ

2025

Съдържание

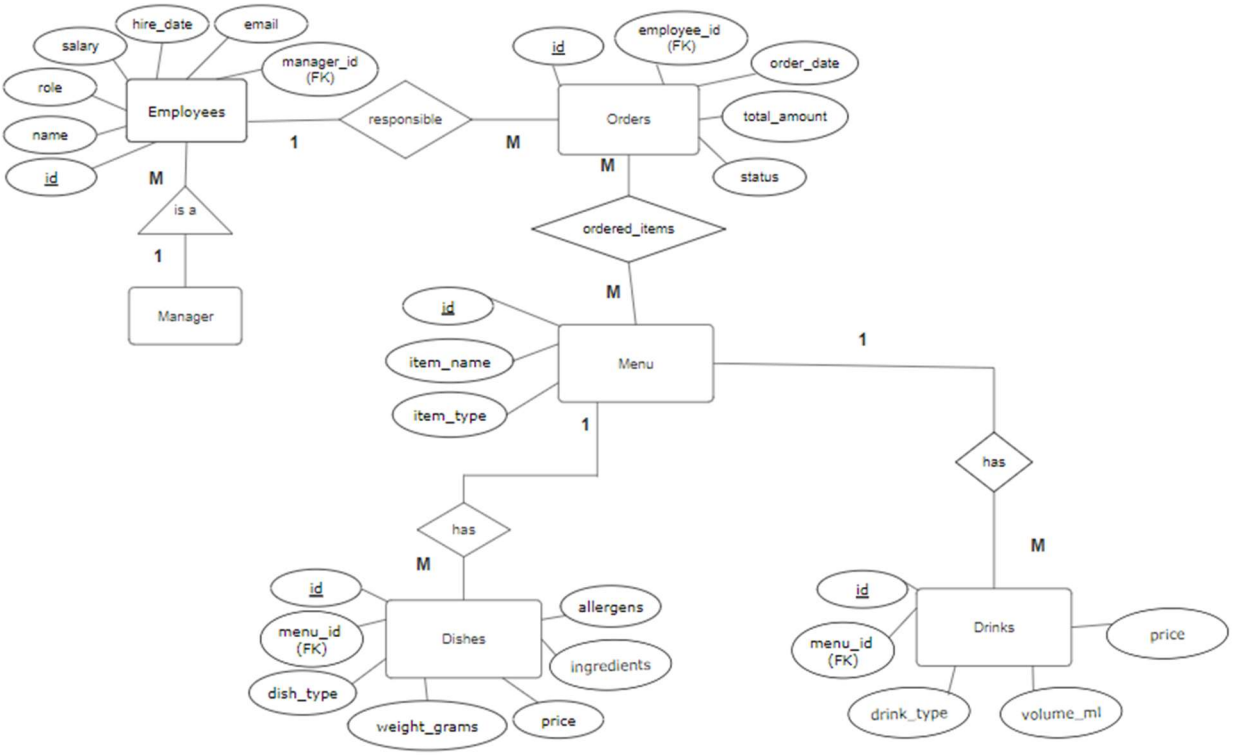
1. Основни цели на курсовия проект

Разработете база от данни за управление на ресторант. В ресторанта има няколко вида служители – главен готвач, обикновени готвачи и помощници, хигиенисти, бармани, сервитьори и мениджър. Ресторантът има меню, което се състои от различни ястия и напитки. Напитките са два вида – алкохолни и безалкохолни. Имат цена, наименование и т.н. Ястията се делят на три основни вида – предястия, основни и десерти. За всяко ястие се съхраняват: наименование, време за приготвяне, цена, използвани продукти, алергени, както и грамаж.

Допълнителни условия:

1. Да се проектира база от данни и да се представи ER диаграма със съответни CREATE TABLE заявки за средата MySQL.
   2. Напишете заявка, в която демонстрирате SELECT с ограничаващо условие по избор.
   3. Напишете заявка, в която използвате агрегатна функция и GROUP BY по ваш избор.
   4. Напишете заявка, в която демонстрирате INNER JOIN по ваш избор.
   5. Напишете заявка, в която демонстрирате OUTER JOIN по ваш избор.
   6. Напишете заявка, в която демонстрирате вложен SELECT по ваш избор.
   7. Напишете заявка, в която демонстрирате едновременно JOIN и агрегатна функция.
   8. Създайте тригер по ваш избор.
   9. Създайте процедура, в която демонстрирате използване на курсор.

2. ER диаграма



1. Проектиране на базата данни

```
CREATE DATABASE restaurant_management;

USE restaurant_management;


CREATE TABLE Employees (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    role ENUM('Head Chef', 'Chef', 'Assistant', 'Cleaner', 'Bartender', 'Waiter', 'Manager'),
    salary DECIMAL(10, 2) NOT NULL,
    hire_date DATE NOT NULL,
```

```
    email VARCHAR(100) NOT NULL UNIQUE,
    manager_id INT,
    FOREIGN KEY (manager_id) REFERENCES Employees(id) ON DELETE SET NULL
);

CREATE TABLE Menu (
    id INT AUTO_INCREMENT PRIMARY KEY,
    item_name VARCHAR(100) NOT NULL,
    item_type ENUM('Dish', 'Drink')
);

CREATE TABLE Dishes (
    id INT AUTO_INCREMENT PRIMARY KEY,
    menu_id INT NOT NULL,
    dish_type ENUM('Appetizer', 'Main Course', 'Dessert'),
    preparation_time INT NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    weight_grams INT NOT NULL,
    ingredients TEXT NOT NULL,
    allergens TEXT NOT NULL,
    FOREIGN KEY (menu_id) REFERENCES Menu(id) ON DELETE CASCADE
);
```

```
CREATE TABLE Drinks (
    id INT AUTO_INCREMENT PRIMARY KEY,
    menu_id INT NOT NULL,
    drink_type ENUM('Alcoholic', 'Non-alcoholic'),
    price DECIMAL(10,2) NOT NULL,
    volume_ml INT NOT NULL,
    FOREIGN KEY(menu_id) REFERENCES Menu(id) ON DELETE CASCADE
);

CREATE TABLE Orders (
    id INT AUTO_INCREMENT PRIMARY KEY,
    employee_id INT NOT NULL,
    order_date DATETIME NOT NULL,
    total_amount DECIMAL(10, 2) NOT NULL,
    status ENUM('Received', 'In Preparation', 'Ready', 'Paid', 'Cancelled') NOT NULL DEFAULT 'Received',
    FOREIGN KEY (employee_id) REFERENCES Employees(id)
);

CREATE TABLE Ordered_Items (
    id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT NOT NULL,
    item_id INT NOT NULL,
    quantity INT NOT NULL,
```

```
    FOREIGN KEY (order_id) REFERENCES Orders(id),
    FOREIGN KEY (item_id) REFERENCES Menu(id));
```

2	09:33:29	CREATE DATABASE restaurant_management	1 row(s) affected
3	09:33:29	USE restaurant_management	0 row(s) affected
4	09:33:29	CREATE TABLE Employees (id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100) NOT N...	0 row(s) affected
5	09:33:29	CREATE TABLE Menu (id INT AUTO_INCREMENT PRIMARY KEY, item_name VARCHAR(100) NOT N...	0 row(s) affected
6	09:33:29	CREATE TABLE Dishes (id INT AUTO_INCREMENT PRIMARY KEY, menu_id INT, dish_type ENUM(...	0 row(s) affected
7	09:33:29	CREATE TABLE Drinks (id INT AUTO_INCREMENT PRIMARY KEY, menu_id INT, drink_type ENUM...	0 row(s) affected
8	09:33:29	CREATE TABLE Orders (id INT AUTO_INCREMENT PRIMARY KEY, employee_id INT, order_date D...	0 row(s) affected
9	09:33:29	CREATE TABLE Ordered_Items (id INT AUTO_INCREMENT PRIMARY KEY, order_id INT, item_id IN...	0 row(s) affected

```
INSERT INTO Employees (name, role, salary, hire_date, email, manager_id) VALUES
('John Peterson', 'Manager', 3000.00, '2021-01-15', 'john.manager@example.com', NULL),
('Maria Smith', 'Head Chef', 2600.00, '2021-03-01', 'maria.chef@example.com', 1),
('George Johnson', 'Waiter', 1200.00, '2022-06-12', 'george.waiter@example.com', 1),
('Anna Davis', 'Waiter', 1300.00, '2023-02-25', 'anna.bartender@example.com', 1),
('Peter Williams', 'Chef', 2100.00, '2022-05-10', 'peter.chef@example.com', 2),
('Sylvia Brown', 'Assistant', 1000.00, '2023-04-05', 'sylvia.assistant@example.com', 2),
('Boris Thompson', 'Cleaner', 900.00, '2023-07-12', 'boris.cleaner@example.com', 1),
('Elena White', 'Waiter', 1150.00, '2023-08-08', 'elena.waiter@example.com', 1),
('Nick Green', 'Bartender', 1350.00, '2024-01-11', 'nick.bartender@example.com', 1),
('Rachel Adams', 'Assistant', 980.00, '2024-02-28', 'rachel.assistant@example.com', 2);
```

id	name	role	salary	hire_date	email	manager_id
1	John Peterson	Manager	3000.00	2021-01-15	john.manager@example.com	NULL
2	Maria Smith	Head Chef	2600.00	2021-03-01	maria.chef@example.com	1
3	George Johnson	Waiter	1200.00	2022-06-12	george.waiter@example.com	1
4	Anna Davis	Waiter	1300.00	2023-02-25	anna.bartender@example.com	1
5	Peter Williams	Chef	2100.00	2022-05-10	peter.chef@example.com	2
6	Sylvia Brown	Assistant	1000.00	2023-04-05	sylvia.assistant@example.com	2
7	Boris Thompson	Cleaner	900.00	2023-07-12	boris.cleaner@example.com	1
8	Elena White	Waiter	1150.00	2023-08-08	elena.waiter@example.com	1
9	Nick Green	Bartender	1350.00	2024-01-11	nick.bartender@example.com	1
10	Rachel Adams	Assistant	980.00	2024-02-28	rachel.assistant@example.com	2

INSERT INTO Menu (item_name, item_type) VALUES

('Caesar Salad', 'Dish'),

('Margherita Pizza', 'Dish'),

('Chocolate Mousse', 'Dish'),

('Coca-Cola', 'Drink'),

('Red Wine', 'Drink'),

('Pumpkin Soup', 'Dish'),

('Spaghetti Bolognese', 'Dish'),

('Tiramisu', 'Dish'),

('Fanta Orange', 'Drink'),

('Zagorka Beer', 'Drink');

id	item_name	item_type
1	Caesar Salad	Dish
2	Margherita Pizza	Dish
3	Chocolate Mousse	Dish
4	Coca-Cola	Drink
5	Red Wine	Drink
6	Pumpkin Soup	Dish
7	Spaghetti Bolognese	Dish
8	Tiramisu	Dish
9	Fanta Orange	Drink
10	Zagorka Beer	Drink

INSERT INTO Dishes (menu_id, dish_type, preparation_time, price, weight_grams, ingredients, allergens) VALUES

(1, 'Appetizer', 10, 7.50, 200, 'Lettuce, chicken, croutons, parmesan, dressing', 'gluten, eggs, dairy'),

(2, 'Main Course', 15, 9.90, 300, 'Dough, tomato sauce, mozzarella, basil', 'gluten, dairy'),

(3, 'Dessert', 5, 5.50, 150, 'Chocolate, cream, eggs', 'eggs, dairy'),

(6, 'Appetizer', 12, 6.00, 250, 'Pumpkin, cream, spices', 'dairy'),

(7, 'Main Course', 18, 11.50, 350, 'Spaghetti, minced beef, tomato sauce', 'gluten'),

(8, 'Dessert', 7, 6.80, 180, 'Mascarpone, coffee, ladyfingers', 'gluten, dairy, eggs');

id	menu_id	dish_type	preparation_time	price	weight_grams	ingredients	allergens
1	1	Appetizer	10	7.50	200	Lettuce, chicken, croutons, parmesan, dressing	gluten, eggs, dairy
2	2	Main Course	15	9.90	300	Dough, tomato sauce, mozzarella, basil	gluten, dairy
3	3	Dessert	5	5.50	150	Chocolate, cream, eggs	eggs, dairy
4	6	Appetizer	12	6.00	250	Pumpkin, cream, spices	dairy
5	7	Main Course	18	11.50	350	Spaghetti, minced beef, tomato sauce	gluten
6	8	Dessert	7	6.80	180	Mascarpone, coffee, ladyfingers	gluten, dairy, eggs

INSERT INTO Drinks (menu_id, drink_type, price, volume_ml) VALUES

(4, 'Non-alcoholic', 2.50, 330),

(5, 'Alcoholic', 4.90, 150),

(9, 'Non-alcoholic', 2.70, 330),

(10, 'Alcoholic', 3.90, 500);

id	menu_id	drink_type	price	volume_ml
1	4	Non-alcoholic	2.50	330
2	5	Alcoholic	4.90	150
3	9	Non-alcoholic	2.70	330
4	10	Alcoholic	3.90	500

INSERT INTO Orders (employee_id, order_date, total_amount, status) VALUES

(4, '2025-04-09 12:30:00', 19.90, 'Paid'),

(8, '2025-04-09 13:10:00', 7.40, 'Ready'),

(3, '2025-04-09 13:40:00', 22.30, 'Paid'),

(8, '2025-04-09 14:10:00', 12.00, 'Cancelled'),

(3, '2025-04-09 14:45:00', 17.60, 'In Preparation'),

(4, '2025-04-09 15:15:00', 9.90, 'Received'),

(3, '2025-04-09 15:50:00', 18.80, 'Ready'),

(8, '2025-04-09 16:20:00', 15.30, 'Paid'),

(3, '2025-04-09 17:00:00', 10.50, 'Paid'),

(8, '2025-04-09 17:45:00', 20.00, 'Received');

id	employee_id	order_date	total_amount	status
1	4	2025-04-09 12:30:00	19.90	Paid
2	8	2025-04-09 13:10:00	7.40	Ready
3	3	2025-04-09 13:40:00	22.30	Paid
4	8	2025-04-09 14:10:00	12.00	Cancelled
5	3	2025-04-09 14:45:00	17.60	In Preparation
6	4	2025-04-09 15:15:00	9.90	Received
7	3	2025-04-09 15:50:00	18.80	Ready
8	8	2025-04-09 16:20:00	15.30	Paid
9	3	2025-04-09 17:00:00	10.50	Paid
10	8	2025-04-09 17:45:00	20.00	Received

INSERT INTO Ordered_Items (order_id, item_id, quantity) VALUES

(1, 2, 1),

(2, 1, 1),

(2, 4, 1),

(3, 10, 1),

(4, 2, 1),

(5, 6, 1),

(5, 5, 1),

(6, 2, 1),

(7, 7, 1),

(8, 1, 1),

(8, 5, 1),

(9, 3, 1),

(9, 9, 1),

(10, 2, 2),

(10, 8, 1);

id	order_id	item_id	quantity
1	1	2	1
2	2	1	1
3	2	4	1
4	3	10	1
5	4	2	1
6	5	6	1
7	5	5	1
8	6	2	1
9	7	7	1
10	8	1	1
11	8	5	1
12	9	3	1
13	9	9	1
14	10	2	2
15	10	8	1

4. SELECT с ограничаващо условие

- *Заявка, която извлича всички служители, чиято заплата е по-висока от 2000*

  SELECT id, name, role, salary FROM Employees WHERE salary > 2000;

id	name	role	salary
1	John Peterson	Manager	3000.00
2	Maria Smith	Head Chef	2600.00
5	Peter Williams	Chef	2100.00

5. Агрегатна функция и GROUP BY

- *Заявка, която показва средната заплата по роля в ресторанта*

```
SELECT role, ROUND(AVG(salary), 2) AS average_salary
FROM Employees
GROUP BY role
ORDER BY average_salary DESC;
```

role	average_salary
Manager	3000.00
Head Chef	2600.00
Chef	2100.00
Bartender	1350.00
Waiter	1216.67
Assistant	990.00
Cleaner	900.00

6. INNER JOIN

- Заявка, която представя всички ястия от менюто, които са основно

```
SELECT menu.item_name, dishes.price FROM menu
INNER JOIN dishes ON dishes.menu_id=menu.id
WHERE dishes.dish_type='Main Course';
```

item_name	price
Margherita Pizza	9.90
Spaghetti Bolognese	11.50

7. OUTER JOIN

- *Заявка, която представя всички служители и техните поръчки дори ако нямат такива*

```
SELECT employees.name, orders.id AS order_id, orders.order_date,
orders.total_amount

FROM employees

LEFT JOIN orders ON employees.id = orders.employee_id;
```

name	order_id	order_date	total_amount
John Peterson	NULL	NULL	NULL
Maria Smith	NULL	NULL	NULL
George Johnson	3	2025-04-09 13:40:00	22.30
George Johnson	5	2025-04-09 14:45:00	17.60
George Johnson	7	2025-04-09 15:50:00	18.80
George Johnson	9	2025-04-09 17:00:00	10.50
Anna Davis	1	2025-04-09 12:30:00	19.90
Anna Davis	6	2025-04-09 15:15:00	9.90
Peter Williams	NULL	NULL	NULL
Sylvia Brown	NULL	NULL	NULL
Boris Thompson	NULL	NULL	NULL
Elena White	2	2025-04-09 13:10:00	7.40
Elena White	4	2025-04-09 14:10:00	12.00
Elena White	8	2025-04-09 16:20:00	15.30
Elena White	10	2025-04-09 17:45:00	20.00
Nick Green	NULL	NULL	NULL
Rachel Adams	NULL	NULL	NULL

8. Вложен SELECT

- *Заявка, която показва всички служители, които са под управлението на мениджъра John Peterson*

```
SELECT name, role
FROM Employees
WHERE manager_id  IN(
   SELECT id
   FROM Employees
   WHERE name = 'John Peterson'
);
```

	name	role
▶	Maria Smith	Head Chef
	George Johnson	Waiter
	Anna Davis	Waiter
	Boris Thompson	Cleaner
	Elena White	Waiter
	Nick Green	Bartender

9. JOIN и агрегатна функция

- *Заявка, която връща списък със служителите и общата сума на поръчките, които те са обработили*

```
SELECT employees.name AS Employee,
SUM(orders.total_amount) AS total_orders_amount
FROM employees
JOIN orders ON employees.id = orders.employee_id
WHERE orders.status='Paid'
GROUP BY employees.id
ORDER BY total_orders_amount DESC;
```

	Employee	total_orders_amount
▶	George Johnson	32.80
	Anna Davis	19.90
	Elena White	15.30

10. Тригери

- *Тригер, който предотвратява въвеждането на отрицателна заплата в таблицата Employees. Ако някой се опита да въведе отрицателна заплата, то заплатата на работника става 0.*

```
DELIMITER |
CREATE TRIGGER BeforeSalaryPaymentsInsert BEFORE INSERT ON employees
FOR EACH ROW
BEGIN
IF(NEW.salary<0) THEN
SET NEW.salary = 0;
END IF;
END |
DELIMITER ;
```

```
INSERT INTO Employees (name, role, salary, hire_date, email) VALUE
('John Moe', 'Cleaner', -1000.00, '2023-01-15', 'john.moe@example.com');
```

id	name	role	salary	hire_date	email	manager_id
1	John Peterson	Manager	3000.00	2021-01-15	john.manager@example.com	NULL
2	Maria Smith	Head Chef	2600.00	2021-03-01	maria.chef@example.com	1
3	George Johnson	Waiter	1200.00	2022-06-12	george.waiter@example.com	1
4	Anna Davis	Waiter	1300.00	2023-02-25	anna.bartender@example.com	1
5	Peter Williams	Chef	2100.00	2022-05-10	peter.chef@example.com	2
6	Sylvia Brown	Assistant	1000.00	2023-04-05	sylvia.assistant@example.com	2
7	Boris Thompson	Cleaner	900.00	2023-07-12	boris.cleaner@example.com	1
8	Elena White	Waiter	1150.00	2023-08-08	elena.waiter@example.com	1
9	Nick Green	Bartender	1350.00	2024-01-11	nick.bartender@example.com	1
10	Rachel Adams	Assistant	980.00	2024-02-28	rachel.assistant@example.com	2
11	John Moe	Cleaner	0.00	2023-01-15	john.moe@example.com	NULL

Забележка: В последния ред на таблицата наблюдаваме как при опит за въвеждане на отрицателна стойност в полето „salary“ автоматично се въвежда 0 с помощта на създадения тригер.

11. Използване на курсор

- *Актуализация на цените на ястията*

```
DELIMITER |
CREATE PROCEDURE UpdateDishPrices(IN increase_percent DECIMAL(5,2))
BEGIN
DECLARE finished INT DEFAULT 0;
DECLARE dish_id INT;
DECLARE dish_name VARCHAR(100);
DECLARE current_price DECIMAL(10,2);

DECLARE dish_cursor CURSOR FOR
SELECT dishes.id, menu.item_name, dishes.price
FROM dishes
JOIN menu ON dishes.menu_id = menu.id;

DECLARE CONTINUE HANDLER FOR NOT FOUND SET finished=1;
CREATE TEMPORARY TABLE temp_dish_prices(
name VARCHAR(100),
adjusted_price DECIMAL(10,2)
) ENGINE = MEMORY;

OPEN dish_cursor;

update_price: WHILE(finished=0) DO
FETCH dish_cursor INTO dish_id, dish_name, current_price;
IF (finished=1) THEN
 LEAVE update_price;
END IF;

UPDATE Dishes
SET price = current_price * (1 + increase_percent / 100)
WHERE id = dish_id;

INSERT INTO temp_dish_prices(name, adjusted_price)
VALUES (dish_name, current_price * (1 + increase_percent / 100));
END WHILE;

CLOSE dish_cursor;
```

```
SELECT * FROM temp_dish_prices;
DROP TEMPORARY TABLE temp_dish_prices;

END |
DELIMITER ;

CALL UpdateDishPrices(10);
```

name	adjusted_price
Caesar Salad	8.25
Margherita Pizza	10.89
Chocolate Mousse	6.05
Pumpkin Soup	6.60
Spaghetti Bolognese	12.65
Tiramisu	7.48